

Zewail City of Science, Technology, and Innovation
University of Science and Technology
Communications and Information Engineering



AI-Driven Intelligent Load Management System for Optimized Resource Allocation in Distributed Networks

Prepared By

Tasneem Muhammed Attia 202101031

Nada Nabil Soudi 202101220

Rghda Salah Ahmed 202101510

Supervised By

Dr. Tamer Ashour

Dr. Doaa Shawky

Contents

1	Executive Summary	3
1.1	System Responsibilities	3
1.2	External APIs	3
1.3	Internal APIs	3
2	API Surface Overview	4
2.1	External Control-Plane APIs	4
2.2	Internal Service APIs	4
2.3	Event-Driven Contracts	4
3	External Control-Plane API	4
3.1	Common API Conventions	4
3.2	Error Format	5
3.3	External API Endpoints	5
3.4	Example External Responses	6
4	Internal Service APIs	7
4.1	Telemetry Ingestion API	7
4.2	Backend Lifecycle API	8
4.3	Routing Controller API	9
4.4	Autoscaler API	10
4.5	Policy Distribution API	10
5	Decision Signals and Control Bus Contracts	11
5.1	Channel Naming Convention	11
5.2	Message Envelope	11
5.3	Forecast Signal Example	12
5.4	Anomaly Signal Example	12
5.5	Routing Recommendation Example	13
5.6	Decision Merge Semantics	13
6	Metrics Storage and Query Model	13
7	Data Models	14
7.1	Policy	14
7.2	Backend Instance	15
7.3	Forecast Result	15
7.4	Anomaly Event	16
7.5	Scaling Action	16
8	Service Responsibilities	16
9	Versioning and Compatibility	17
10	Security and Operational Conventions	18

11 Assumptions and Open Issues	18
11.1 Service Discovery	18
11.2 Routing Update Mechanism	18
11.3 Policy Persistence	19
11.4 Experiment Management	19
11.5 Future Extensions	19

1 Executive Summary

SmartLoad is a middleware platform that sits between clients and backend services and improves routing, resilience, and resource utilization through telemetry-driven decision making.

1.1 System Responsibilities

SmartLoad performs the following high-level functions:

- Accept and route incoming client traffic through the ingress layer
- Collect telemetry from the load balancer and backend services
- Persist and expose metrics for analysis and observability
- Generate routing and scaling decisions through multiple decision engines
- Apply policy constraints and safe-mode overrides
- Update NGINX routing state and infrastructure capacity through controller components

1.2 External APIs

SmartLoad exposes a RESTful control-plane API with the following properties:

- Base path: `/api/v1`
- Data format: JSON
- Intended consumers: operator dashboard, CLI tools, admin scripts, and automation systems
- Error handling using Problem Details JSON
- Resource-oriented endpoints for policies, backends, routing state, scaling state, anomalies, and experiments

1.3 Internal APIs

Communication between internal components uses multiple protocols because different interactions have different latency and coupling requirements:

- OTLP/gRPC for telemetry ingestion
- REST for internal control and lifecycle operations
- Redis Pub/Sub for asynchronous decision and actuation messages
- SQL for metrics storage and decision-engine queries

2 API Surface Overview

SmartLoad exposes three communication categories.

2.1 External Control-Plane APIs

These APIs are intended for operators, dashboards, CLI tooling, and automation scripts. They are synchronous, authenticated, and resource-oriented.

2.2 Internal Service APIs

These APIs are used between SmartLoad services for lifecycle, configuration, and control operations that are better modeled as request/response interactions.

2.3 Event-Driven Contracts

These are asynchronous contracts published on Redis Pub/Sub. They decouple the decision engines from controllers and allow multiple services to subscribe to the same control signals.

3 External Control-Plane API

The external API is a RESTful interface located at:

`/api/v1`

These APIs allow operators and automation tools to configure, observe, and evaluate the SmartLoad system.

3.1 Common API Conventions

- JSON payloads encoded in UTF-8
- Standard HTTP methods: GET, POST, PUT, PATCH, DELETE
- Resource identifiers passed as path parameters
- UTC timestamps in ISO 8601 format
- Idempotency for selected POST operations through Idempotency-Key
- ETags supported for selected updateable resources

3.2 Error Format

All error responses follow the Problem Details structure.

```
{
  "type": "string",
  "title": "string",
  "status": 400,
  "detail": "human-readable error",
  "instance": "request-id-or-path"
}
```

3.3 External API Endpoints

Endpoint	Method	Description
System and Health		
/api/v1/status	GET	Returns system version, active mode, and high-level health summary.
/api/v1/health	GET	Liveness endpoint for the control plane.
/api/v1/ready	GET	Readiness endpoint checking major dependencies.
/api/v1/system/metrics	GET	Returns summarized runtime KPIs for dashboards and admin scripts.
Policy Management		
/api/v1/policies	GET	Lists available policies or the active policy set.
/api/v1/policies	POST	Creates a new policy definition.
/api/v1/policies/{policy_id}	GET	Retrieves one policy document.
/api/v1/policies/{policy_id}	PUT	Replaces an entire policy resource.
/api/v1/policies/{policy_id}	PATCH	Updates selected policy fields.
/api/v1/policies/{policy_id}/activate	POST	Activates a policy for live enforcement.
/api/v1/policies/safe-mode	POST	Enables or disables safe mode.

Endpoint	Method	Description
Backend Inventory and Topology		
/api/v1/backends	GET	Lists registered backend instances and their current state.
/api/v1/backends/{backend_id}	GET	Returns detailed information about one backend instance.
/api/v1/backends/{backend_id}/quarantine	POST	Forces a backend into operator-controlled quarantine.
/api/v1/backends/{backend_id}/restore	POST	Restores a quarantined backend to operator-managed state.
Routing, Scaling, and Anomalies		
/api/v1/routing/state	GET	Returns current routing configuration and effective upstream weights.
/api/v1/scaling/state	GET	Returns current and desired capacity state.
/api/v1/anomalies	GET	Lists detected anomalies and backend health signals.
/api/v1/forecasts/latest	GET	Returns the latest published workload forecast.
Experiments and Evaluation		
/api/v1/experiments	POST	Starts a benchmark or evaluation run.
/api/v1/experiments	GET	Lists experiment runs and their status.
/api/v1/experiments/{experiment_id}	GET	Returns metadata and results for one experiment.

3.4 Example External Responses

Example response for GET /api/v1/status:

```
{
  "version": "1.0.0",
  "mode": "hybrid",
  "safe_mode": false,
  "health": "degraded",
  "active_backends": 6,
  "control_bus": "connected"
}
```

Example response for GET `/api/v1/routing/state`:

```
{
  "algorithm": "hybrid",
  "generated_at": "2026-03-16T13:25:00Z",
  "upstreams": [
    {"backend_id": "backend-1", "weight": 0.45, "state": "active"},
    {"backend_id": "backend-2", "weight": 0.35, "state": "active"},
    {"backend_id": "backend-3", "weight": 0.20, "state": "degraded"}
  ]
}
```

4 Internal Service APIs

Internal communication between SmartLoad services uses secure internal endpoints. These APIs align with the service interactions visible in the component, channel, and sequence diagrams.

4.1 Telemetry Ingestion API

Telemetry data is received through OTLP/gRPC and may also be represented through the following logical ingestion paths:

- POST `/internal/v1/telemetry/metrics:ingest`
- POST `/internal/v1/telemetry/logs:ingest`
- POST `/internal/v1/telemetry/traces:ingest`

Expected behavior:

- Input accepted asynchronously
- Response status: HTTP 202 Accepted for buffered processing

- Request producer: NGINX and instrumented backend or telemetry collectors
- Request consumer: telemetry collector or telemetry service

Example payload:

```
{
  "source": "nginx",
  "emitted_at": "2026-03-15T12:00:00Z",
  "metrics": [
    {
      "name": "http_requests_total",
      "labels": {"upstream": "orders", "status": "200"},
      "points": [
        {"t": "2026-03-15T12:00:00Z", "v": 120}
      ]
    }
  ]
}
```

4.2 Backend Lifecycle API

The backend inventory must be updated whenever new instances are created, removed, quarantined, or restored.

Endpoint	Method	Description
/internal/v1/backends/register	POST	Registers a newly provisioned backend instance with routing metadata.
/internal/v1/backends/{backend_id}/deregister	POST	Removes a backend from the active registry.
/internal/v1/backends	GET	Returns the internal backend registry used by routing controllers.
/internal/v1/backends/{backend_id}/health	PATCH	Updates backend health or quarantine state.

Example registration payload:

```
{
  "backend_id": "backend-12",
  "address": "10.0.1.42",
  "port": 8080,
  "zone": "local-dev",
  "capacity_weight": 1.0,
  "state": "active"
}
```

4.3 Routing Controller API

The routing controller converts effective decisions into concrete NGINX upstream updates.

Endpoint	Method	Description
/internal/v1/routing/update	POST	Applies a new set of routing weights or rankings to the controller.
/internal/v1/routing/state	GET	Returns controller-visible routing state.
/internal/v1/routing/reload	POST	Triggers an NGINX configuration reload when needed.

Example routing update payload:

```
{
  "generated_at": "2026-03-16T13:27:00Z",
  "mode": "hybrid",
  "upstreams": [
    {"backend_id": "backend-1", "weight": 0.50},
    {"backend_id": "backend-2", "weight": 0.35},
    {"backend_id": "backend-3", "weight": 0.15}
  ]
}
```

4.4 Autoscaler API

The autoscaler executes capacity actions after consuming forecast and policy signals.

Endpoint	Method	Description
/internal/v1/autoscaler/scale-up	POST	Requests scale-out by a target number of instances or target capacity.
/internal/v1/autoscaler/scale-down	POST	Requests scale-in while respecting policy constraints.
/internal/v1/autoscaler/state	GET	Returns autoscaler state, cooldown timers, and desired capacity.
/internal/v1/autoscaler/actions/{action_id}	GET	Returns status for one scaling action.

Example scale-up payload:

```
{
  "reason": "forecast_exceeds_capacity",
  "desired_instances": 8,
  "min_instances": 3,
  "max_instances": 12
}
```

4.5 Policy Distribution API

The policy manager may distribute policies through Redis, shared configuration, or a small internal API.

Endpoint	Method	Description
/internal/v1/policy/current	GET	Returns the active policy snapshot for internal services.
/internal/v1/policy/reload	POST	Forces services to reload or re-fetch policy state.
/internal/v1/policy/validate	POST	Validates policy values before activation.

5 Decision Signals and Control Bus Contracts

SmartLoad uses Redis Pub/Sub to distribute decision and actuation signals. This is necessary because the forecasting engine, RL engine, anomaly detector, routing controller, autoscaler, and policy manager must remain decoupled while still sharing a consistent control picture.

5.1 Channel Naming Convention

Channel naming format:

`smartload.<topic>.v1`

Typical channels include:

- `smartload.policy.v1`
- `smartload.forecast.v1`
- `smartload.routing.v1`
- `smartload.anomaly.v1`
- `smartload.scaling.v1`
- `smartload.backend.lifecycle.v1`

5.2 Message Envelope

All messages share a common envelope.

```
{
  "schema_version": "1.0.0",
  "message_id": "msg_123",
  "type": "smartload.forecast.v1",
  "emitted_at": "2026-03-15T12:01:00Z",
  "source": {"service": "forecast-engine"},
  "payload": {}
}
```

5.3 Forecast Signal Example

```
{
  "schema_version": "1.0.0",
  "message_id": "msg_forecast_001",
  "type": "smartload.forecast.v1",
  "emitted_at": "2026-03-15T12:01:00Z",
  "source": {"service": "forecast-engine"},
  "payload": {
    "window": "5m",
    "predicted_rps": 2400,
    "confidence": 0.87
  }
}
```

5.4 Anomaly Signal Example

```
{
  "schema_version": "1.0.0",
  "message_id": "msg_anomaly_012",
  "type": "smartload.anomaly.v1",
  "emitted_at": "2026-03-15T12:01:05Z",
  "source": {"service": "anomaly-detection-engine"},
  "payload": {
    "backend_id": "backend-3",
    "status": "degraded",
    "reason": "latency_spike",
    "confidence": 0.92
  }
}
```

5.5 Routing Recommendation Example

```
{
  "schema_version": "1.0.0",
  "message_id": "msg_route_044",
  "type": "smartload.routing.v1",
  "emitted_at": "2026-03-15T12:01:10Z",
  "source": {"service": "rl-engine"},
  "payload": {
    "mode": "hybrid",
    "scores": [
      {"backend_id": "backend-1", "score": 0.74},
      {"backend_id": "backend-2", "score": 0.18},
      {"backend_id": "backend-3", "score": 0.08}
    ]
  }
}
```

5.6 Decision Merge Semantics

The final effective action follows this hierarchy:

1. Apply anomaly constraints and remove or down-weight unhealthy nodes.
2. Apply active policy constraints such as safe mode and min/max limits.
3. Use RL routing scores when available and permitted.
4. Fall back to classical load balancing when intelligent outputs are unavailable or blocked.
5. Use forecasts to drive scaling decisions, not per-request routing.

6 Metrics Storage and Query Model

Time-series metrics are stored using TimescaleDB, a PostgreSQL extension optimized for time-series workloads.

Representative raw table:

```
metrics_raw(  
  time TIMESTAMPTZ,  
  service TEXT,  
  instance TEXT,  
  metric TEXT,  
  labels JSONB,  
  value DOUBLE PRECISION  
)
```

Representative aggregate query:

```
SELECT time_bucket('60s', time) AS t,  
       avg(value) AS avg_value  
FROM metrics_raw  
WHERE metric = 'http_requests_total'  
       AND service = 'nginx'  
GROUP BY t  
ORDER BY t;
```

Representative decision-engine query pattern:

```
SELECT time, instance, value  
FROM metrics_raw  
WHERE metric IN ('latency_p95_ms', 'cpu_utilization_pct')  
       AND time >= now() - interval '5 minutes';
```

7 Data Models

7.1 Policy

```
{
  "policy_id": "policy-default",
  "name": "default-hybrid-policy",
  "mode": "hybrid",
  "safe_mode": false,
  "routing_constraints": {
    "min_instances": 2,
    "max_instances": 10,
    "max_change_pct": 20
  },
  "slo": {
    "p95_latency_ms": 200,
    "error_rate_pct": 1.0
  },
  "rl": {
    "enabled": true,
    "exploration_rate": 0.05
  }
}
```

7.2 Backend Instance

```
{
  "backend_id": "backend-3",
  "address": "10.0.1.10",
  "port": 8080,
  "state": "active",
  "health": "degraded",
  "capacity_weight": 1.0,
  "registered_at": "2026-03-15T11:58:00Z"
}
```

7.3 Forecast Result

```
{
  "window": "5m",
  "predicted_rps": 2400,
  "lower_bound": 2200,
  "upper_bound": 2650,
  "confidence": 0.87,
  "generated_at": "2026-03-15T12:01:00Z"
}
```

7.4 Anomaly Event

```
{
  "backend_id": "backend-3",
  "status": "degraded",
  "reason": "latency_spike",
  "confidence": 0.92,
  "observed_at": "2026-03-15T12:01:05Z"
}
```

7.5 Scaling Action

```
{
  "action_id": "scale-20260315-001",
  "direction": "up",
  "desired_instances": 8,
  "reason": "forecast_exceeds_capacity",
  "status": "in_progress"
}
```

8 Service Responsibilities

Service	Responsibility
API Gateway / NGINX	Accepts client traffic, forwards requests to backends, exposes health endpoints, and applies effective upstream routing configuration.

Service	Responsibility
Telemetry Collector	Receives OTLP telemetry, batches and normalizes payloads, and forwards processed telemetry downstream.
Telemetry Service	Validates and stores processed metrics, and provides queryable telemetry to other services.
Metrics Database	Persists raw and aggregated metrics for observability, forecasting, anomaly detection, and evaluation.
Forecasting Engine	Predicts near-future workload and publishes forecast signals to the control bus.
RL Engine	Computes adaptive routing scores or rankings and publishes them for controller consumption.
Anomaly Detection Engine	Detects degraded or unhealthy nodes and publishes anomaly flags.
Classical Load Balancing Engine	Provides deterministic fallback routing logic when learning-based control is unavailable or disabled.
Policy Manager	Stores, validates, activates, and distributes policy state and safe-mode overrides.
NGINX Routing Controller	Merges effective routing instructions and applies upstream updates to the load balancer.
Autoscaler	Decides and executes scale-up and scale-down actions under policy constraints.
Container Runtime API	Starts or stops backend instances and exposes infrastructure lifecycle hooks.

9 Versioning and Compatibility

The SmartLoad API follows explicit versioning practices.

- Major API version is embedded in the path, for example `/api/v1` and `/internal/v1`.
- Breaking changes require a new major version.
- Backward-compatible field additions are allowed within the same version.
- Redis event contracts include `schema_version`.

- Services should ignore unknown fields when safe to do so.
- Migration periods should support old and new contracts in parallel where practical.

10 Security and Operational Conventions

- External control-plane endpoints should be authenticated and authorized.
- Internal endpoints should be reachable only on trusted internal networks.
- Secrets such as database credentials and runtime API tokens must be provided through environment variables or secret stores.
- Control-bus publishers should use a stable service identity in the message envelope.
- Audit logs should record policy activation, safe-mode changes, scaling actions, and manual operator interventions.

11 Assumptions and Open Issues

Several implementation details require clarification or final engineering decisions.

11.1 Service Discovery

Backend discovery is required because autoscaling can add and remove instances dynamically.

Possible approaches include:

- Polling the Docker runtime or container runtime API
- Maintaining an internal registry updated by backend lifecycle endpoints
- Introducing a dedicated service registry in future versions

11.2 Routing Update Mechanism

The design assumes that the routing controller can apply upstream changes to NGINX through one of the following:

- Regenerating configuration and reloading NGINX
- Updating a dynamic upstream source
- Using an adapter layer between control signals and NGINX runtime configuration

11.3 Policy Persistence

The policy manager is assumed to persist policy documents for auditability and restart recovery. The storage mechanism can be a relational database, versioned files, or another configuration store.

11.4 Experiment Management

The experiment endpoints are included because SmartLoad is both a research project and a benchmarked prototype. Final implementation may keep them as internal-only endpoints if operator exposure is unnecessary.

11.5 Future Extensions

Potential future improvements include:

- Multi-tenant policy isolation
- Rate limiting and admission control APIs
- Retry and dead-letter support for internal control messages
- Stronger service-to-service authentication
- Rich operator UI with full runtime control